

Machine-Level Data Representation

Binary and hexadecimal representation, signed values, two's complement, endianness, and alignment

Dr. J. Burns

Topic 3 Outline

Topic Objective

Analysis of how machine-level information is encoded, interpreted, and organized across fixed-width registers and byte-addressed memory structures.

- **Section 1:** Bits, Bytes, Words, and Base Conversions
- **Section 2:** Binary and Hexadecimal Representation
- **Section 3:** Unsigned and Signed Integer Representations
- **Section 4:** Two's Complement Encoding and Arithmetic Behavior
- **Section 5:** Memory Endianness and Data Alignment in AArch64
- **Section 6:** Summary and Worked Self-Test Exercises

Section 1

Bits, Bytes, Words, and Base Conversions

Why Data Representation Matters

All machine-level information is stored as fixed-width bit patterns:

- **Context Dependent:** The same bit pattern can mean different things under different interpretations (e.g., `0xFF` can be 255 unsigned or -1 signed).
- **ISA Rules:** Software interpretation determines whether bits represent data items, memory addresses, or machine instructions.
- **Fixed-Width Bounds:** Registers impose strict hardware width limits on numeric ranges.
- **Layout Rules:** Memory organization dictates how multi-byte values are mapped and retrieved.

Data Units in Memory and Registers

Hierarchical Units of Storage

- **Bit:** A single binary digit (0 or 1).
- **Byte:** Contains 8 bits; the foundational unit of addressable memory.
- **Halfword:** Commonly 16 bits (2 bytes).
- **Word:** Commonly 32 bits (4 bytes) in standard architectures.
- **Doubleword:** Commonly 64 bits (8 bytes), native to AArch64 registers (x0–x30).

Example

One AArch64 register width = 64 bits = 8 bytes.

The Binary Number System (Base 2)

Binary maps directly to physical dual-state digital hardware:

- Each digit position represents a distinct power of two.
- **LSB (Least Significant Bit)**: The rightmost bit position (2^0).
- **MSB (Most Significant Bit)**: The leftmost bit position in a fixed-width container.

Positional Evaluation Example

$$1011_2 = (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1) = 11_{10}$$

Base Conversion Mechanics

Binary to Decimal

- Write power-of-two weights above each bit ($2^0, 2^1, 2^2 \dots$).
- Disregard positions where the bit is 0.
- Sum the weights where the bit is 1.
- Example:
 $01010110_2 = 64 + 16 + 4 + 2 = 86_{10}$.

Decimal to Binary

- Repeatedly divide the decimal value by 2.
- Record each remainder as a binary digit.
- Read remainders from bottom to top.
- Pad with leading zeros to match container width.
- Example: $13_{10} = 1101_2$.

Section 2

Binary and Hexadecimal Representation

The Hexadecimal Number System (Base 16)

Hexadecimal provides a compact, readable shorthand for machine-level data:

- Digits range from 0 to 9 and letters *A* to *F* (representing values 10 to 15).
- **Exact Mapping:** Each single hexadecimal digit maps precisely to 4 binary bits ($2^4 = 16$).
- Used universally for memory addresses, instruction opcodes, and register dumps.

Example

$$0x2A = 0010\ 1010_2 = (2 \times 16) + 10 = 42_{10}$$

Direct Binary-Hexadecimal Conversion

Converting between binary and hex requires no intermediate decimal math:

Binary to Hexadecimal

- Group binary bits into sets of four starting from the right (LSB).
- Pad the leftmost group with leading zeros if necessary.
- Map each 4-bit nibble directly to its hex digit.
- Example: $1101\ 0110_2 = 0xD6$.

Hexadecimal to Binary

- Take each individual hex character.
- Expand it into its exact 4-bit binary equivalent.
- Concatenate the nibbles together.
- Example: $0x5F = 0101\ 1111_2$.

AArch64 Registers and Data Widths

AArch64 general-purpose registers are 64 bits wide, supporting flexible data sizing:

- A 64-bit register (x0–x30) holds 64 bits of raw data.
- The exact same bit pattern can be interpreted as signed integers, unsigned integers, pointers, or masks.
- Memory instructions (ldr, str) explicitly specify access sizes such as bytes, halfwords, words, and doublewords.

Example Interpretation

Setting `x0 = 0xFFFFFFFFFFFFFFFF` can represent the unsigned integer 18,446,744,073,709,551,615 or the signed integer `-1`.

Section 3

Unsigned and Signed Integer Representations

Unsigned Integer Representation

Unsigned integers represent only non-negative values (0 and positive integers):

- Every bit position contributes a strictly positive power-of-two weight.
- The leftmost bit (MSB) acts as a high-order magnitude bit, not a sign indicator.
- An n -bit unsigned container spans the exact range from 0 to $2^n - 1$.

Unsigned Range Examples

- 8-bit unsigned range: 0 to $2^8 - 1 = 255$.
- 64-bit unsigned range: 0 to $2^{64} - 1 \approx 1.84 \times 10^{19}$.
- Example: $1111\ 1111_2 = 255$ in 8 bits.

Unsigned Arithmetic and Fixed-Width Limits

Unsigned operations are evaluated modulo 2^n for an n -bit container:

- **Fixed-Width Bound:** When an unsigned addition exceeds the maximum representable value ($2^n - 1$), the fixed-width register cannot expand.
- **Truncation & Modular Behavior:** High-order carry bits exceeding the bit width are truncated, leaving the low n bits as the result modulo 2^n .
- Later topics show how processors can record selected arithmetic facts for conditional instructions.

Classic Overflow Example

In 8-bit unsigned arithmetic:

$$255 + 1 = 1111\ 1111_2 + 1 = 0000\ 0000_2 = 0$$

with a carry out.

Signed Integer Challenges & Sign-Magnitude Failure

Signed integers require encoding both positive and negative values:

- **Naive Approach (Sign-Magnitude):** Dedicate the MSB as a sign flag (0 = positive, 1 = negative).
- **Problems with Sign-Magnitude:**
 - **Dual Zeros:** Encodes both positive zero (00000) and negative zero (10000).
 - **Arithmetic Complication:** Ordinary binary addition cannot simply treat all sign-magnitude bit patterns as fixed-width integers and obtain correct signed arithmetic.

The Engineering Solution

Modern general-purpose processors use **two's-complement representation** for signed integers.

Section 4

Two's Complement Representation and Arithmetic Behavior

Two's Complement Representation

Core Design Objective

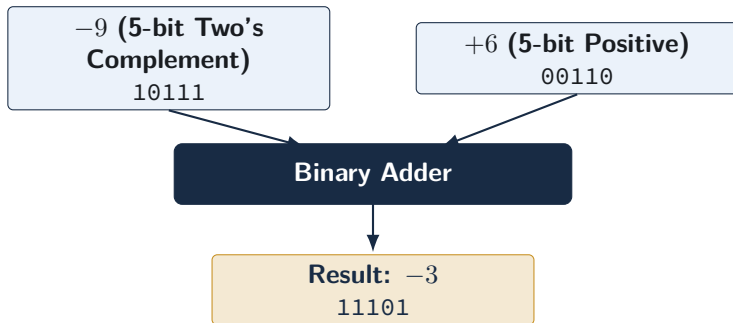
Two's complement allows signed addition and subtraction to use the same fixed-width binary-adder structure used for unsigned arithmetic, while maintaining a single representation for zero.

Algorithm for Negation

- 1 Start with the positive magnitude in binary.
 - 2 **Invert every bit** ($0 \rightarrow 1$, $1 \rightarrow 0$).
 - 3 **Add 1** to the inverted result.
- Example, -5 in 8 bits: $0000\ 0101 \rightarrow 1111\ 1010 \rightarrow 1111\ 1011$.

Two's Complement in Action: $-9 + 6$

Standard fixed-width binary addition produces the correct two's-complement result:



- Adding 10111 (-9) and 00110 ($+6$) yields 11101, which represents -3 .

Two's Complement Numeric Range

An n -bit container under two's-complement encoding spans asymmetric bounds:

- **Total Patterns:** 2^n unique bit combinations.
- **Signed Range:** -2^{n-1} to $2^{n-1} - 1$.
- **Asymmetry:** There is one more negative value than positive value because zero occupies one bit pattern.
- The most negative value has only its most significant bit set ($1000 \dots 0$).

8-Bit and 64-Bit Ranges

- 8-bit two's-complement range: -128 to $+127$.
- 64-bit two's-complement range: $-9,223,372,036,854,775,808$ to $+9,223,372,036,854,775,807$.

Signed Arithmetic Overflow

Definition of Signed Overflow

Signed overflow occurs when a mathematical signed result lies outside the representable range of the destination width.

- For addition, overflow occurs when two operands with the same sign produce a result with the opposite sign.
- Hardware can detect overflow from the operand signs and the result sign.
- Example in 8-bit arithmetic: $127 + 1 = 0111\ 1111 + 0000\ 0001 = 1000\ 0000$. The bit pattern represents -128 , so signed overflow occurred.

Section 5

Memory Endianness and Data Alignment in AArch64

Memory as a Byte-Addressed Structure

AArch64 memory is organized as a byte-addressed structure:

- Each memory address identifies one 8-bit byte.
- Multi-byte values occupy consecutive byte addresses.
- A multi-byte object's address normally identifies its lowest-addressed byte.

Example: 64-Bit Doubleword Layout

A 64-bit value beginning at address 0x1000 occupies byte addresses 0x1000, 0x1001, 0x1002, 0x1003, 0x1004, 0x1005, 0x1006, and 0x1007.

Endianness: Ordering Multi-Byte Values

Definition

Endianness defines the ordering of bytes when a multi-byte value is stored across consecutive memory addresses.

Big-Endian Ordering

- Stores the **most significant byte** at the lowest memory address.
- The remaining bytes follow toward increasing addresses.

Little-Endian Ordering

- Stores the **least significant byte** at the lowest memory address.
- This is the usual configuration encountered in modern AArch64 systems.

Little-Endian Layout in AArch64 Practice

AArch64 implementations commonly operate with little-endian data accesses:

- The numerical value held in a register is conceptually independent of how its bytes are ordered in memory.
- Loads assemble register values from memory bytes according to the configured byte order.
- Stores place register-value bytes into memory according to that byte order.
- Endianness becomes especially visible when examining memory one byte at a time.

Concrete Layout Example

Storing the 64-bit value `0x1234567890ABCDEF` beginning at address `0x2000` in little-endian order gives:

Address	0x2000	0x2001	0x2002	0x2003	0x2004	0x2005	0x2006	0x2007
Byte	0xEF	0xCD	0xAB	0x90	0x78	0x56	0x34	0x12

Data Alignment in AArch64

Natural Alignment

An object is naturally aligned when its starting address is a multiple of its size in bytes.

- **64-Bit Doubleword:** naturally aligned at addresses divisible by 8.
- **32-Bit Word:** naturally aligned at addresses divisible by 4.
- **16-Bit Halfword:** naturally aligned at addresses divisible by 2.
- Natural alignment generally simplifies and can improve memory-system access.
- AArch64 permits many ordinary unaligned data accesses, but behavior and performance can depend on the access type, memory attributes, and implementation.

Section 6

Summary & Self-Test

Topic 3 Comprehensive Summary

- **Fixed-Width Representation:** Information is stored as bit patterns whose meaning depends on interpretation and context.
- **Base Conversions:** Binary (base 2) and hexadecimal (base 16) provide useful notations for machine-level values.
- **Unsigned Integers:** Represent non-negative values from 0 through $2^n - 1$ in an n -bit width.
- **Two's Complement:** Provides the standard fixed-width representation for signed integers and supports signed arithmetic using ordinary binary-adder structures.
- **Endianness & Alignment:** Byte ordering determines how multi-byte values map to memory, while alignment describes the relationship between object size and starting address.

Self-Test: Questions 1 & 2

Question 1: Bits, Words, and Bases

What is the relationship between bits, bytes, words, and 64-bit AArch64 registers? Why is hexadecimal usually easier to read than binary when inspecting machine-level values?

Question 2: Unsigned Arithmetic Limits

What range of values can be represented by an unsigned n -bit integer? What happens when an 8-bit unsigned value of 255 has 1 added to it?

Self-Test: Solutions 1 & 2

Solution 1

A bit is one binary digit; a byte contains 8 bits; in AArch64 terminology a word is 32 bits, while the x general-purpose register view is 64 bits. Hexadecimal is compact because each hex digit maps directly to four binary bits.

Solution 2

An unsigned n -bit container represents values from 0 to $2^n - 1$. For 8 bits, $255 + 1$ produces a low 8-bit result of 0 modulo 256, with carry out.

Self-Test: Questions 3 & 4

Question 3: Two's Complement Negation

How do you form the two's-complement negative of a positive binary number? What signed range is representable in a 32-bit two's-complement integer?

Question 4: Signed vs. Unsigned Interpretation

How can the same 32-bit pattern represent different numerical values when interpreted as signed versus unsigned?

Self-Test: Solutions 3 & 4

Solution 3

To form the two's-complement negative of a representable positive value, invert every bit and add 1. A 32-bit two's-complement integer spans -2^{31} through $2^{31} - 1$.

Solution 4

Unsigned interpretation assigns non-negative powers-of-two weights to all bit positions. Two's-complement interpretation assigns the most significant bit a negative positional weight. Thus `0xFFFFFFFF` represents 4,294,967,295 unsigned but -1 signed.

Self-Test: Questions 5 & 6

Question 5: Endianness in Memory

What is the difference between little-endian and big-endian byte ordering? Which ordering is most commonly encountered in AArch64 systems?

Question 6: Data Alignment

What does natural alignment mean, and why can alignment matter to a memory system?

Self-Test: Solutions 5 & 6

Solution 5

Little-endian places the least significant byte at the lowest address. Big-endian places the most significant byte at the lowest address. Modern AArch64 systems are commonly configured for little-endian data accesses.

Solution 6

Natural alignment means that an object's starting address is a multiple of its size in bytes. Aligned accesses are generally easier for memory hardware to handle efficiently, while unaligned-access behavior and cost can depend on the access and implementation.

Review: Summary Table of Data Types

Representation Type	Bit Interpretation	8-Bit Example / Range
Unsigned Integer	All bits have non-negative positional weights	0 to 255
Signed Two's Complement	MSB has negative positional weight	-128 to +127
Hexadecimal Notation	Each digit represents 4 bits	0x00 to 0xFF
Little-Endian Memory	Least significant byte stored at lowest address	Byte-ordering rule

Wrap-Up and Next Steps

Transition to Topic 4

- **Integer Arithmetic and Performance Basics:** Evaluating fundamental ALU operations alongside processor timing metrics.
- **Integer Addition, Subtraction, and Shifts:** How ALUs execute core arithmetic and bitwise shifting instructions.
- **Multiplication and Division:** Hardware and architectural treatment of scaling operations and division.
- **Arithmetic Status:** How later instructions can use arithmetic results to control program flow.
- **Performance Metrics:** Evaluating clock rate, CPI, and execution time.

Topic 3 Source Notes

Source Reference Verification

- Representation concepts in this topic establish the numerical and memory-layout foundation needed for later AArch64 programming.
- Two's-complement arithmetic, byte ordering, and alignment will recur in later instruction, memory, and datapath topics.